Every stream call builds a pipeline and drops it

Bytes and objects per call, match at element 500, compact headers on.

Operation	JDK stream	Objects	EC, every version
`anyMatch` / `anySatisfy`	176 B	7	0 B
`filter().findFirst()` / `detect`	216 B	8	0 B
`filter().count()` / `count`	224 B	8	0 B

- The stream’s own cost is per call: the same at element 1 as at 1000.
- EC versions: capturing or fat-free, eager or lazy, all 0 B.

Where the bytes come from

`list.stream().filter(p).findFirst()`, one call, match at element 500.

Origin	Objects	Bytes
Stream source: `ReferencePipeline$Head`, spliterator	2	80 B
`filter` stage and its sink	2	72 B
terminal op: `FindSink`, `Optional`	2	32 B
caller lambda `Integer.valueOf(test)::equals`	1	16 B
`Integer` box, outside the cache	1	16 B
Total per call	**8**	**216 B**

From bytes per call to spikes

Bytes per call are measured; rates and spikes are derived.

Element 500	B per call	MB/s at 10 M calls/s	10 s spike
JDK `anyMatch`	176	1,760	17.6 GB
JDK `findFirst`	216	2,160	21.6 GB
JDK `count`	224	2,240	22.4 GB
EC, every version	0	0	0